LSTP Specification 1.0 -- 07-13-2026

Datacenter Platform Software

13 July, 2026

Table of Contents

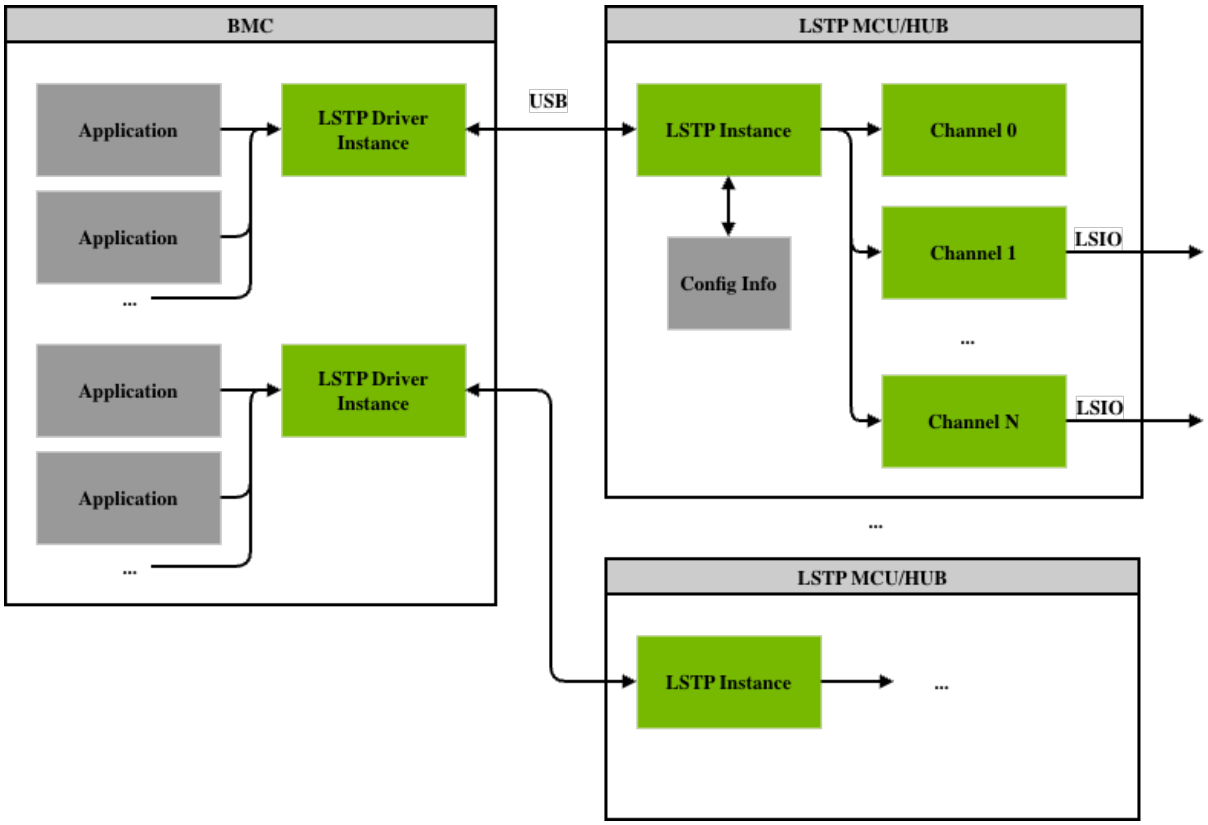
1 Protocol Overview 4
1.1 LSTP Configuration Overview 5
2 USB Configuration 7
2.1 OCP Recovery 7
3 LSTP Packet Format 8
3.1 Header 8
3.1.1 Status Codes 8
3.2 Channel Types 9
4 Channel-specific details 10
4.1 Management Channel (Channel 0) 10
4.1.1 Channel Config Format 10
4.1.2 Supported Commands 10
4.1.3 Channel Flags: 12
4.2 Flashrom (SPI) Channel 13
4.2.1 Channel Config Format 13
4.2.2 Supported Commands 13
4.3 GPIO channel 15
4.3.1 Channel Config Format 15
4.3.2 Supported Commands 19
4.4 I2C channel 23
4.4.1 Channel Config Format 23
4.4.2 Supported Commands 23
4.5 UART channel 25
4.5.1 Channel Config Format 25
4.5.2 Supported Commands 26
4.6 IPMI channel 27
4.6.1 Channel Config Format 27
4.6.2 Supported Commands 27
4.7 Vendor Defined MMIO 27

4.7.1 Channel Config Format 28
4.7.2 Supported Commands 28
5 Revision History 29

1 Protocol Overview

LSTP (Low-Speed Transport Protocol) is a protocol that tunnels low-speed I/O over USB between a Linux host and an LSTP endpoint device. The goal is to enable robust, standard-based communication between platform management controllers and host systems using USB as the transport for LSTP.

Each low-speed interface is contained within individual channels on the device endpoint. An LSTP host has one driver instance for each LSTP device. There can be multiple channels of the same I/O type (I2C, GPIO banks, SPI, etc).



Channels only support 1 outstanding command (no concurrent commands), with an exception that interrupt packets can come in the middle of a pending command. Channels are independent, so commands on different channels may execute concurrently.

Each channel must implement all commands defined for its channel type. (Devices should not implement deprecated commands)

The LSTP device must return a response for an outstanding request within 100ms to meet driver timing requirements (if the command requires a response).

Configuration options are fusable after programming through a command.

1.1 LSTP Configuration Overview

LSTP devices are configured per channel through the Management Channel (Channel 0). Each channel's configuration is stored in nonvolatile memory, so it persists through resets and power cycles, and can be permanently fused/locked so it cannot be rewritten.

Configuration is flexible: not only the settings within a channel, but also the number of channels and even the channel types can be programmed. A single physical device may therefore be reconfigured into entirely different layouts.

The Management Channel (always at index 0) serves two purposes:

- **Discovery:** The host reads the LSTP config to learn the device's channels and capabilities (`READ_CONFIG`).
- **Provisioning:** The host writes channel configs (`WRITE_CONFIG`) and can optionally fuse them so they're permanently locked (`LOCK_CONFIGS`).

For full command details, see Section 4.1.

Multifunction pin devices: Where pins can serve more than one function (e.g. GPIOs, I2C, or UART), not every channel configuration combination is physically valid. Such devices are expected to document their supported configuration combinations so the host programs only valid setups. As an example, one LSTP device supports five flashable modes of operation:

Configuration Modes
No I/O Channels (Only Management Channel)
1 GPIO Channel (16 Pins)
1 GPIO Channel (14 Pins) + 1 UART Channel
1 GPIO Channel (14 Pins) + 1 I2C Channel
1 GPIO Channel (12 Pins) + 1 UART Channel + 1 I2C Channel

Best Effort Principal: For `WRITE_CONFIG` , the device should put its "best effort" into implementing the configurations written to it. A device may or may not support writing to any/all config values.

If a value is not supported, the device should either:

- Set it to the closest reasonable value possible, or
- Return `LSTP_NOT_SUPP` if it cannot support the option at all.

The host is assumed to have prior knowledge of what the device supports from its datasheet or product specification.

Example — `OUTPUT_DRIVE_STRENGTH`**:**

Suppose a GPIO's hardware only supports three discrete drive levels. 50 mA, 100 mA, and 500 mA. If the host writes 120 mA, the device can round it to the nearest supported level (100 mA) and apply that. A follow-up READ_CONFIG then reports 100 mA, so the host can see the value that was actually applied.

2 USB Configuration

LSTP packets may be larger than a single USB packet. The portion of the payload that is larger than a USB packet will come in subsequent USB packets. Multiple LSTP packets can fit in a single USB packet.

LSTP Driver will bind to the following USB descriptors:

<table>
<tr><th>Descriptor</th><th>Value</th></tr>
<tr><td>bInterfaceClass</td><td>255</td></tr>
<tr><td>bInterfaceSubClass</td><td>63</td></tr>
<tr><td>bInterfaceProtocol</td><td>1
(We track the LSTP version with protocol)</td></tr>
<tr><td>bNumEndpoints</td><td>2
<table><tr><th>Endpoints</th><th>Size</th><th>Purpose</th></tr><tr><td>Bulk In</td><td>Minimum 64 Bytes</td><td>Device-to-host LSTP responses and data</td></tr><tr><td>Bulk Out</td><td>Minimum 64 bytes</td><td>Host-to-device LSTP commands and data</td></tr></table></td></tr>
<tr><td>bcdDevice</td><td>Shall be used to track chip/device revision - increment on ROM change. Currently unused by LSTP driver.</td></tr>
</table>

2.1 OCP Recovery

Programmable firmware must support OCP recovery over USB. https://www.opencompute.org/documents/ocp-recovery-document-1p1-final-pdf

3 LSTP Packet Format

LSTP data will be in little endian format.

3.1 Header

Size = 4B

Header is followed by a channel/command specific payload.

Byte (bit)		Field	Description
1B		Channel ID	LSTP channel identifier
1B	[0:6]	Command/Status	Command = for requests Status = for response
	[7]	Request/Response	0 = Request 1 = Response
2B		Payload Length	Payload Length = N For an unsupported command the device shall skip N bytes

3.1.1 Status Codes

Value	Name	Description
0x00	LSTP_SUCCESS	Command completed successfully
0x01	LSTP_ERROR	Invalid command or parameter
0x02	LSTP_TIMEOUT	Command timed out
0x03	LSTP_BUSY	LSTP link busy, retry later
0x04	LSTP_NACK	LSTP link error or not established
0x05	LSTP_ARB_LOST	Resource allocation failure
0x06	LSTP_NOT_SUPP	Device does not support this command or option

Value	Name	Description
0x07	LSTP_TOO_LARGE	Requested size too large (device implementation constraint)

3.2 Channel Types

Channel Type	Description
0x00	Management Channel (Channel 0)
0x01	Flashrom (SPI) Channel
0x02	GPIO
0x03	I2C master
0x04	UART
0x05	IPMI
0x06	Vendor Defined MMIO
0x07 - 0x7E	Reserved
0x7F - 0xFF	OEM Defined

4 Channel-specific details

4.1 Management Channel (Channel 0)

The Management Channel functionality is documented in Section 1. It is always channel index 0. The channel itself has a configuration for negotiating LSTP version, providing the number of channels.

4.1.1 Channel Config Format

Size	Field
1B	LSTP version
1B	Num channels = N (Does not include Management Channel)

4.1.2 Supported Commands

Configurations are read and written to in blocks that span from [*offset* : min(*read/write_length*, *config_max_offset*)]. A full channel config might not fit in a single packet. The base fields (channel type, channel flags, and channel name) are always present in every write request and read response. If the host requests a config read beyond the maximum offset for that channel, the device will return as much data as is available.

Config offsets 0x7FFF-0xFFFF are reserved for custom vendor defined channel logic.

Management Channel (Channel 0) name is used as the LSTP Interface Name (Ex: "My_Board_LSTP_Hub").

Code	Command	Request Format	Response Format
0x00-0x07	Reserved (Flashrom Backwards Compatibility) https://www.flashrom.org/		

<table>
<tr><th>Code</th><th>Command</th><th>Request Format</th><th>Response Format</th></tr>
<tr><td>0x08</td><td>READ_CONFIG</td><td>
<table>
<tr><th>Byte</th><th>Field</th></tr>
<tr><td>4B</td><td>LSTP header</td></tr>
<tr><td>1B</td><td>Channel ID</td></tr>
<tr><td>2B</td><td>Offset ¹</td></tr>
<tr><td>2B</td><td>Read length</td></tr>
</table>
</td><td>
<table>
<tr><th>Byte</th><th>Field</th></tr>
<tr><td>4B</td><td>LSTP header</td></tr>
<tr><td>1B</td><td>Channel type</td></tr>
<tr><td>1B</td><td>Channel flags
(See Channel Flags Below)</td></tr>
<tr><td>16B</td><td>Channel name</td></tr>
<tr><td>M *
1B</td><td>Channel config
(See each channel for custom format)</td></tr>
</table>
</td></tr>
<tr><td>0x09</td><td>WRITE_CONFIG ¹</td><td>
<table>
<tr><th>Byte</th><th>Field</th></tr>
<tr><td>4B</td><td>LSTP header</td></tr>
<tr><td>1B</td><td>Channel ID</td></tr>
<tr><td>2B</td><td>Offset</td></tr>
<tr><td>1B</td><td>Channel type</td></tr>
<tr><td>1B</td><td>Channel flags
(See channel flags below)</td></tr>
<tr><td>16B</td><td>Channel name</td></tr>
<tr><td>M *
1B</td><td>Channel config
(See each channel for custom format)</td></tr>
</table>
</td><td>
<table>
<tr><th>Byte</th><th>Field</th></tr>
<tr><td>4B</td><td>LSTP header</td></tr>
</table>
</td></tr>
</table>

<table>
<tr><th>Code</th><th>Command</th><th>Request Format</th><th>Response Format</th></tr>
<tr><td>0x0A</td><td>WRITE_CONFIG_MODE</td><td><table><tr><th>Byte</th><th>Field</th></tr><tr><td>4B</td><td>LSTP header</td></tr><tr><td>1B</td><td>Lock State
0x00 = Non-volatile (To Flash/OTP)
0x01 = Fused/ Locked
0x02 = Volatile (To RAM)</td></tr></table></td><td><table><tr><th>Byte</th><th>Field</th></tr><tr><td>4B</td><td>LSTP header</td></tr></table></td></tr>
<tr><td>0x0B</td><td>READ_CONFIG_MODE</td><td><table><tr><th>Byte</th><th>Field</th></tr><tr><td>4B</td><td>LSTP header</td></tr></table></td><td><table><tr><th>Byte</th><th>Field</th></tr><tr><td>4B</td><td>LSTP header</td></tr><tr><td>1B</td><td>Lock State
0x00 = Non-volatile (To Flash/OTP)
0x01 = Fused/Locked
0x02 = Volatile (To RAM)</td></tr></table></td></tr>
<tr><td>0x10-0x FF</td><td>Reserved
(Flashrom Backwards Compatibility)
https:// www.flashrom.org/</td><td></td><td></td></tr>
</table>

4.1.3 Channel Flags:

Flag Bit	Explanation	Values
0	Channel Enable – Whether the channel is enabled or not	0 = Disabled 1 = Enabled
1:7	RESERVED	N/A

4.2 Flashrom (SPI) Channel

Compatible with Flashrom style SPI commands. Has some reserved command codes to preserve backwards compatibility with Flashrom. See: https://www.flashrom.org/

4.2.1 Channel Config Format

Size	Field	Options
1B	Number of devices (Should be number of CS pins)	
4B	Clock Speed (Hz)	
2B	Output Drive Strength (mA)	0x0000 = Not specified 0x0001 = 1mA (Weakest) 0xFFFF = 65535mA (Strongest)
2B	Slew Rate (0.1 ns increments)	0x0000 = Not specified 0x0001 = 0.1ns (Fastest) 0xFFFF ≈ 6554ns (Slowest)
2B	Input Debounce Time (µs)	0x0000 = Not specified 0x0001 = 1µs (Shortest) 0xFFFF = 65535µs (Longest)
2B	Glitch Filter Width (ns)	0x0000 = Not specified 0x0001 = 1ns (Shortest) 0xFFFF = 65535ns (Longest)

4.2.2 Supported Commands

Flashrom commands are only the lower 4 bits of the command byte.
The upper 3 indicate what CS pins to use.

Code [0:3]	Command	Request Format	Response Format
0x0	Reserved (Flashrom Backwards Compatibility)		

<table>
<tr><th>Code [0:3]</th><th>Command</th><th>Request Format</th><th>Response Format</th></tr>
<tr><td>0x1</td><td>READ</td><td><table><tr><th>Byte</th><th>Field</th></tr><tr><td>4B</td><td>LSTP header</td></tr><tr><td>4B</td><td>Read Length (N)</td></tr></table></td><td><table><tr><th>Byte</th><th>Field</th></tr><tr><td>4B</td><td>LSTP header</td></tr><tr><td>1B * N</td><td>Read Data</td></tr></table></td></tr>
<tr><td>0x2</td><td>WRITE</td><td><table><tr><th>Byte</th><th>Field</th></tr><tr><td>4B</td><td>LSTP header</td></tr><tr><td>1B * N</td><td>Write Data</td></tr></table></td><td><table><tr><th>Byte</th><th>Field</th></tr><tr><td>4B</td><td>LSTP header</td></tr></table></td></tr>
<tr><td>0x3</td><td>WRITE_READ</td><td><table><tr><th>Byte</th><th>Field</th></tr><tr><td>4B</td><td>LSTP header</td></tr><tr><td>1B * N</td><td>Write Data</td></tr></table></td><td><table><tr><th>Byte</th><th>Field</th></tr><tr><td>4B</td><td>LSTP header</td></tr><tr><td>1B * N (same length as write data)</td><td>Read Data</td></tr></table></td></tr>
<tr><td>0x4</td><td>Reserved (Flashrom Backwards Compatibility)</td><td></td><td></td></tr>
</table>

Upper 4 command bits:

Bit	Value
[0:3]	Command (See above)
[4:5]	Set CS State Bit Flags (For CS pins selected below) 0 = Keep CS state the same from last transaction 0b01 = Deassert after transaction 0b10 = Assert before transaction (Will toggle pin if already asserted) 0b11 = Assert before and deassert after transaction
[6]	CS Select 0 = CS0 1 = CS1
[7]	Request/Response Bit

4.3 GPIO channel

4.3.1 Channel Config Format

4.3.1.1 Channel Config Block

Note: Since channel configs are read/written to in blocks of memory, the “total number of GPIOs” field is the first byte of the blob, and would only be included if the config read/write includes byte (offset) 0.

Each channel config block supports up to 255 GPIOs (1 byte). For more than 255, additional channel config blocks are appended to the back. (See: *Multiple GPIO Channel Blocks below*)

Size	Field
1B	Total number of GPIOs (N) (Max 255)

Size	Field
48B * N	GPIO Config (1 to N) **Size** / **Field** 32B / GPIO Name 16B / GPIO pin config options (See table below)

4.3.1.2 GPIO Pin Options

For WRITE_CONFIG, the device should put its “best effort” into implementing the configurations written to it. A device may or may not support writing to any/all config values. If a value is not supported, the device should set to the closest reasonable value possible, or return LSTP_NOT_SUPP.

See: *linux/include/linux/pinctrl/pinconf-generic.h*

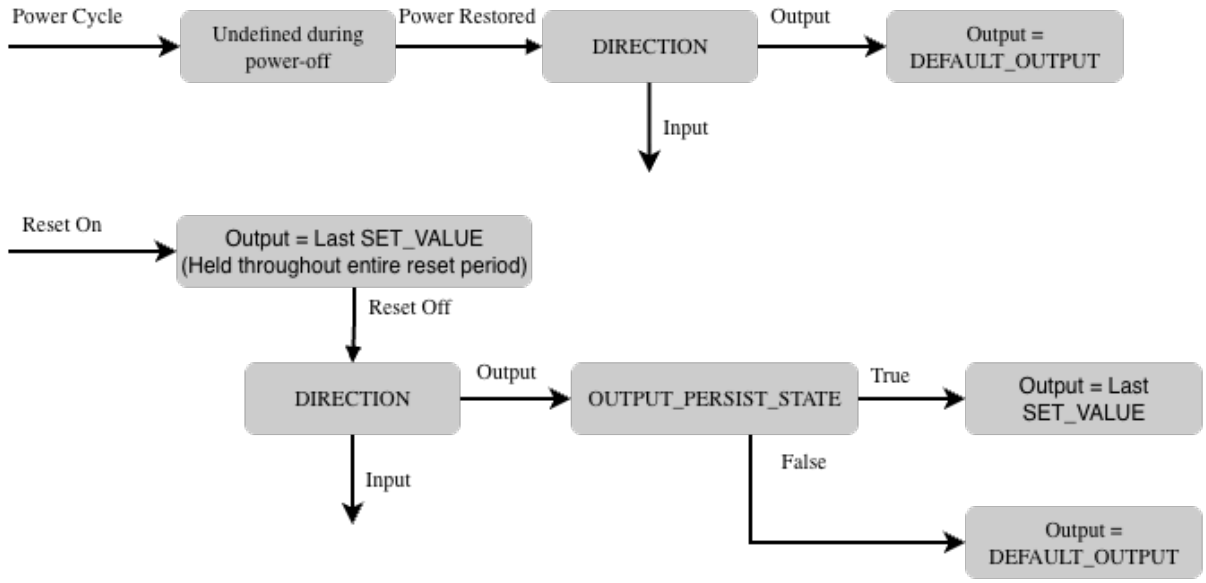
Config Option	Config Offset Bytes (Size)	Config Value - 2B
DIRECTION	0x0 (1B)	0x00 = Output 0x01 = Input
DEFAULT_OUTPUT	0x1 (1B)	0x00 = Low 0x01 = High 0x02 = Z
OUTPUT_DRIVE_CONFIG	0x2 (1B)	0x00 = Push/Pull (Hi <> Lo) 0x01 = Open Drain (Z <> Lo) 0x02 = Open Source (Hi <> Z)
OUTPUT_PERSIST_STATE See: *GPIO State Persistence* below	0x3 (1B)	0x00 = False 0x01 = True
BIAS_PULL_CONFIG	0x4 (1B)	0x00 = No Pull 0x01 = Pull Up 0x02 = Pull Down
BIAS_PULL_STRENGTH	0x5 (2B)	Value in 16Ω 0x0000 = Not specified 0x0001 = 16Ω (Strongest) 0xFFFF ≈ 1MΩ (Weakest)

Config Option	Config Offset Bytes (Size)	Config Value - 2B
OUTPUT_DRIVE_STRENGTH	0x7 (2B)	Drive strength in mA 0x0000 = Not specified 0x0001 = 1mA (Weakest) 0xFFFF = 65535mA (Strongest)
SLEW_RATE	0x9 (2B)	Value in 0.1 ns 0x0000 = Not specified 0x0001 = 0.1ns (Fastest) 0xFFFF ≈ 6554ns (Slowest)
INPUT_DEBOUNCE_TIME	0xB (2B)	Debounce time in µs 0x0000 = Not specified 0x0001 = 1µs (Shortest) 0xFFFF = 65535µs (Longest)
LED_SUPPORT	0XD (1B)	Whether this GPIO supports LED functions 0x00 = Not LED 0x01 = Hold (pattern brightness jumps between steps) 0x02 = Interpolate (pattern brightness interpolates steps)
RESERVED	0xE-0xF (2B)	Set to 0U.

4.3.1.3 GPIO State Persistence

OUTPUT_PERSIST_STATE governs whether the GPIO output setting from the “SET_VALUE” command will persist through a device reset. (No glitches or transient state changes are allowed)

Recommended logic for OUTPUT_PERSIST_STATE supporting GPIOs:



4.3.1.4 Multiple GPIO Channel Blocks

To support more than 255 GPIOs, multiple channel config blocks are appended together. There can be a max of 255 config blocks for a total of 65,025 GPIOs.

Size	Field
1B + 48B * N_1	Config 1 (See: *Channel Config Block above*)
1B + 48B * N_2	Config 2
1B + 48B * N_n	Config N

4.3.2 Supported Commands

<table>
<tr><th>Code</th><th>Command</th><th>Request Format</th><th>Response Format</th></tr>
<tr><td>0x00</td><td>GET_VALUE</td><td>
<table>
<tr><th>Byte</th><th>Field</th></tr>
<tr><td>4B</td><td>LSTP header</td></tr>
<tr><td>2B * N
(N inferred from payload length)</td><td>Pin offset 0 to N</td></tr>
</table>
</td><td>
<table>
<tr><th>Byte</th><th>Field</th></tr>
<tr><td>4B</td><td>LSTP header</td></tr>
<tr><td>1B * N
(N inferred from payload length)</td><td>Pin state 0 to N
0x00 = Low
0x01 = High</td></tr>
</table>
</td></tr>
<tr><td>0x01</td><td>SET_VALUE</td><td>
<table>
<tr><th colspan="2">Byte</th><th>Field</th></tr>
<tr><td colspan="2">4B</td><td>LSTP header</td></tr>
<tr><td rowspan="2">3B * N
(N inferred from payload length)</td><td>2B</td><td>GPIO offset</td></tr>
<tr><td>1B</td><td>GPIO state
0x00 = Low
0x01 = High</td></tr>
</table>
</td><td>
<table>
<tr><th>Byte</th><th>Field</th></tr>
<tr><td>4B</td><td>LSTP header</td></tr>
</table>
</td></tr>
</table>

<table>
<tr><th>Code</th><th>Command</th><th>Request Format</th><th>Response Format</th></tr>
<tr><td>0x02</td><td>GET_IRQ_CONFIG</td><td><table><tr><th>Byte</th><th>Field</th></tr><tr><td>4B</td><td>LSTP header</td></tr><tr><td>2B</td><td>Pin offset</td></tr></table></td><td><table><tr><th>Byte</th><th>Field</th></tr><tr><td>4B</td><td>LSTP header</td></tr><tr><td>1B</td><td>0x00 = None (disable)
0x01 = Rising
0x02 = Falling
0x03 = Both
0x04 = High
0x05 = Low</td></tr></table></td></tr>
<tr><td>0x03</td><td>SET_IRQ_CONFIG</td><td><table><tr><th>Byte</th><th>Field</th></tr><tr><td>4B</td><td>LSTP header</td></tr><tr><td>2B</td><td>Pin offset</td></tr><tr><td>1B</td><td>0x00 = None (disable)
0x01 = Rising
0x02 = Falling
0x03 = Both
0x04 = High
0x05 = Low</td></tr></table></td><td><table><tr><th>Byte</th><th>Field</th></tr><tr><td>4B</td><td>LSTP header</td></tr></table></td></tr>
</table>

<table>
<tr><th>Code</th><th>Command</th><th>Request Format</th><th>Response Format</th></tr>
<tr><td>0x04</td><td>IRQ_EVENT</td><td>
<table>
<tr><th colspan="2">Byte</th><th>Field</th></tr>
<tr><td colspan="2">4B</td><td>LSTP header</td></tr>
<tr><td rowspan="2">3B * N
(N inferred from payload length)</td><td>2B</td><td>GPIO N offset</td></tr>
<tr><td>1B</td><td>GPIO N state
0x00 = Low
0x01 = High</td></tr>
</table>
</td><td>N/A</td></tr>
</table>

<table>
<tr><th>Code</th><th>Command</th><th>Request Format</th><th>Response Format</th></tr>
<tr>
<td>0x05</td>
<td>SET_LED_PATTERN</td>
<td>
<table>
<tr><th colspan="3">Byte</th><th>Field</th></tr>
<tr><td colspan="3">4B</td><td>LSTP header</td></tr>
<tr><td rowspan="4">(7B + 5B * M) * N (N inferred from payload length)</td><td colspan="2">2B</td><td>GPIO N offset</td></tr>
<tr><td colspan="2">4B</td><td>Repeats† (0 = repeats forever)</td></tr>
<tr><td colspan="2">1B</td><td>Pattern Steps (M) † (0 = off)</td></tr>
<tr><td rowspan="2">5B * M</td><td>1B</td><td>Brightness</td></tr>
<tr><td>4B</td><td>Time (ms)</td></tr>
</table>
</td>
<td>
<table>
<tr><th>Byte</th><th>Field</th></tr>
<tr><td>4B</td><td>LSTP header</td></tr>
</table>
</td>
</tr>
</table>

† Note: according to Linux ABI, repeat = -1 means forever, N ≥ 1 plays N times (0 and < -1 → -EINVAL); To avoid negative numbers, the LSTP driver maps repeat = -1 to 0.

The "hold at brightness" primitive used by brightness_set uses repeats == 0 && pattern_steps == 1. In this case, since time_ms is not used, the driver sends time_ms = 1 as its default value.

4.4 I2C channel

4.4.1 Channel Config Format

Size	Field	Options
1B	Speed	0 = 100 kHz 1 = 400 kHz 2 = 1 MHz 3 = 3.4 MHz 4...127 = Reserved
2B	Output Drive Strength (mA)	0x0000 = Not specified 0x0001 = 1mA (Weakest) 0xFFFF = 65535mA (Strongest)
2B	Slew Rate (0.1 ns)	(0.1 ns increments) 0x0000 = Not specified 0x0001 = 0.1ns (Fastest) 0xFFFF ≈ 6554ns (Slowest)
2B	Input Debounce Time (µs)	0x0000 = Not specified 0x0001 = 1µs (Shortest) 0xFFFF = 65535µs (Longest)
2B	Glitch Filter Width (ns)	0x0000 = Not specified 0x0001 = 1ns (Shortest) 0xFFFF = 65535ns (Longest)
1B	Bus Low Timeout (ms)	0x00 = No timeout 0x01 = 1ms (Shortest) 0xFF = 256ms (Longest)
1B	Bus Arbitration Retries	0x00 = No Multimaster Support 0x01 = 1 Retry 0xFF = 256 Retries

4.4.2 Supported Commands

To support repeated starts, we reserve bit 6 of the command to indicate whether to send a stop.

<table>
<tr><th>Code</th><th>Command</th><th>Request Format</th><th>Response Format</th></tr>
<tr><td>0x01- Send STOP
0x41 - Don't Send STOP</td><td>READ</td><td><table><tr><th>Byte</th><th>Field</th></tr><tr><td>4B</td><td>LSTP header</td></tr><tr><td>1B</td><td>I2C target address</td></tr><tr><td>2B</td><td>Read length = N</td></tr></table></td><td><table><tr><th>Byte</th><th>Field</th></tr><tr><td>4B</td><td>LSTP header</td></tr><tr><td>N*1B</td><td>Read data</td></tr></table></td></tr>
<tr><td>0x02 - Send STOP
0x42 - Don't send STOP</td><td>WRITE</td><td><table><tr><th>Byte</th><th>Field</th></tr><tr><td>4B</td><td>LSTP header</td></tr><tr><td>1B</td><td>I2C target address</td></tr><tr><td>N*1B</td><td>Write data</td></tr></table></td><td><table><tr><th>Byte</th><th>Field</th></tr><tr><td>4B</td><td>LSTP header</td></tr></table></td></tr>
<tr><td>0x03
0x43</td><td>Deprecated</td><td></td><td></td></tr>
<tr><td>0x04 - Send STOP
0x44 - Don't send STOP</td><td>WRITE_READ (for SMBus protocol emulation)</td><td><table><tr><th>Byte</th><th>Field</th></tr><tr><td>4B</td><td>LSTP header</td></tr><tr><td>1B</td><td>I2C target address</td></tr><tr><td>2B</td><td>Read length = N</td></tr><tr><td>M*1B</td><td>Write data
(M inferred from header length)</td></tr></table></td><td><table><tr><th>Byte</th><th>Field</th></tr><tr><td>4B</td><td>LSTP header</td></tr><tr><td>N*1B</td><td>Read data</td></tr></table></td></tr>
</table>

<table>
<tr><th>Code</th><th>Command</th><th>Request Format</th><th>Response Format</th></tr>
<tr>
<td>0x05 - Send STOP
0x45 - Don’t send STOP</td>
<td>SMBUS_BLOCK_READ</td>
<td>
<table>
<tr><th>Byte</th><th>Field</th></tr>
<tr><td>4B</td><td>LSTP header</td></tr>
<tr><td>1B</td><td>I2C target address</td></tr>
<tr><td>2B</td><td>Additional bytes to read
0 - no PEC
1 - read PEC</td></tr>
<tr><td>M*1B</td><td>Write data
(M inferred from header length)</td></tr>
</table>
</td>
<td>
<table>
<tr><th>Byte</th><th>Field</th></tr>
<tr><td>4B</td><td>LSTP header</td></tr>
<tr><td>N*1B</td><td>Read data</td></tr>
</table>
</td>
</tr>
</table>

4.5 UART channel

4.5.1 Channel Config Format

Size		Field	Options
4B		Speed	Baud rate in Hz
1B		Data Bits	Number of data bits
1B		Stop Bits	Number of stop bits
1B	[0:3]	Parity	0 = None 1 = Odd 2 = Even 3 = Mark 4 = Space
	[4:7]	Flow Control	0 = None 1 = XON/XOFF 2 = RTS/CTS 3 = DSR/DTR

Size	Field	Options
2B	Output Drive Strength (mA)	0x0000 = Not specified 0x0001 = 1mA (Weakest) 0xFFFF = 65535mA (Strongest)
2B	Slew Rate (0.1 ns increments)	0x0000 = Not specified 0x0001 = 0.1ns (Fastest) 0xFFFF ≈ 6554ns (Slowest)
2B	Input Debounce Time (µs)	0x0000 = Not specified 0x0001 = 1µs (Shortest) 0xFFFF = 65535µs (Longest)
2B	Glitch Filter Width (ns)	0x0000 = Not specified 0x0001 = 1ns (Shortest) 0xFFFF = 65535ns (Longest)

4.5.2 Supported Commands

UART is a bidirectional interface so WRITE requests are sent from host to device for TX and device to host for RX.

<table>
<tr><th>Code</th><th>Command</th><th>Request Format</th><th>Response Format</th></tr>
<tr>
<td>0x00
See flag definition below</td>
<td>WRITE</td>
<td>
<table>
<tr><th>Byte</th><th>Field</th></tr>
<tr><td>4B</td><td>LSTP header</td></tr>
<tr><td>N*1B</td><td>Write data</td></tr>
</table>
</td>
<td>
<table>
<tr><th>Byte</th><th>Field</th></tr>
<tr><td>4B</td><td>LSTP header</td></tr>
</table>
Status field in the header is used to indicate ACK/NACK
</td>
</tr>
</table>

Upper 6 bits of the command byte indicate UART operations/events. Device side support for these events is implementation specific (optional). If a flag is not supported by the device it should be ignored without error.

Bit	Value
[7]	Request/Response Bit

Bit	Value
[6]	1 = Carrier down (typically remote Serial/UART interface to BMC)
[5]	1 = Transmit overrun (typically remote Serial/UART interface to BMC)
[4]	1 = Break detected (typically remote Serial/UART interface to BMC)
[3]	1 = Generate Break (typically BMC to remote Serial/UART interface)
[2]	Reserved
[1:0]	Command code

4.6 IPMI channel

4.6.1 Channel Config Format

Empty

4.6.2 Supported Commands

Code	Command	Request Format	Response Format
0x00	IPMI_MESSAGE	Byte / Field 4B / LSTP header N*1B / IPMI message	N/A - no response, symmetrical channel

4.7 Vendor Defined MMIO

Generic support for vendor defined memory layout. Can be used for debug or additional configuration.

4.7.1 Channel Config Format

Vendor Defined Fusable Config Values

4.7.2 Supported Commands

<table>
<tr><th>Code</th><th>Command</th><th>Request Format</th><th>Response Format</th></tr>
<tr><td>0x00</td><td>Read</td><td><table><tr><th>Byte</th><th>Field</th></tr><tr><td>4B</td><td>LSTP header</td></tr><tr><td>4B</td><td>Offset</td></tr><tr><td>2B</td><td>Length = N</td></tr></table></td><td><table><tr><th>Byte</th><th>Field</th></tr><tr><td>4B</td><td>LSTP header</td></tr><tr><td>N*1B</td><td>Data</td></tr></table></td></tr>
<tr><td>0x01</td><td>Write</td><td><table><tr><th>Byte</th><th>Field</th></tr><tr><td>4B</td><td>LSTP header</td></tr><tr><td>4B</td><td>Offset</td></tr><tr><td>N*1B</td><td>Data
(N inferred from header length)</td></tr></table></td><td><table><tr><th>Byte</th><th>Field</th></tr><tr><td>4B</td><td>LSTP header</td></tr></table></td></tr>
</table>

5 Revision History

07 Jul 2026

- Add additional analog config options for serial I/O channels
- Add tri state DEFAULT_OUTPUT mode

25 Jun 2026

- Reworked config locking commands to 3 state modes for finer OTP flash control
- Reworded MMIO to Vendor Defined MMIO

12 Jun 2026

- Clarified multi function pin configurations

11 Jun 2026

- Added bcdDevice usage guidance

02 Jun 2026

- Added READ_LOCK_STATE command

27 May 2026

- Deprecate SPI POSTED_WRITE

18 May 2026

- Removed using config length of 0 for read all. Use full packet size instead.

15 May 2026

- Added LED control config and command for GPIOs

12 May 2026

- Reverted GPIO commands to use 2 byte offsets
- Allow for multiple GPIO config blocks to support more than 255 GPIOs

06 May 2026

- Updated diagram for OUTPUT_PERSIST_STATE
- Clarified read/write config logic
- Clarified config fusing/locking logic

27 Apr 2026

- Add TOO_LARGE error code
- Limit Flashrom to 2 CS pins

22 Apr 2026

- Offset I2C commands by +1

- Clarified that channel configurations must be persistent and fusable
- Clarified the OUTPUT_PERSIST_STATE option for the GPIO channel

16 Apr 2026

- Add SMBUS_BLOCK_READ command, mark READ_RECVLEN as deprecated

08 Apr 2026

- Aligned GPIO commands to use 1B offset index, like the channel config
- Removed custom buffer size – only use USB descriptor packet size
- Added USB path programming for custom device behavior or naming

23 Mar 2026

- Updated UART channel to align with OBMF-ICP

19 Mar 2026

- Clarified CS pin states for Flashrom commands

06 Mar 2026

- Added speed and CS pin number config item for SPI
- Allow multiple GPIO IRQ events in a single packet

26 Feb 2026

- Updated requirements for OCP recovery over USB
- Reformatted the LSTP protocol and channel descriptions
- Changed language to clarify GPIO configs must contain all options

24 Feb 2026

- Changed GPIO get/set commands to allow for multiple pin requests at once
- Added optional Management channel config option to extend large LSTP packets over multiple USB packets

11 Feb 2026

- Added requirement for 100ms response turnaround from LSTP device

06 Feb 2026

- Allows for variable packet size (minimum 64 bytes)

27 Jan 2026

- Clarified language for LSTP packets that span over multiple USB packets

22 Jan 2026

- Removed I2C bus recovery
- Put statement allowing multiple LSTP packets into a single USB packet

12 Jan 2026

- Added IPMI channel definition
- Added per-channel enable/disable (needed for IPMI/SSIF)
- Changed Management channel to be backwards compatible with Flashrom commands
- Changed Chunk ID to offset for multipacket blobs

12 Dec 2025

- Moved GPIO IRQ to be runtime configurable
- Fixed SLEW_RATE size to 2B
- Split configs into chunks for when it overruns packet size
- Defined unspecified value to some GPIO config options
- Changed GPIO config format to support chunks
- Changed GPIO names to 32 bytes long

10 Dec 2025

- Merged all channel configs into a Management Channel command
- Made per channel defined config FRU format
- Removed all config related commands from channels
- Added protocol overview + example diagram
- Defined units for some GPIO config options